

Tokenizers — Complete Notes

0. What a tokenizer is and why it exists

Models only understand **numbers**, so a tokenizer's job is to turn **text** → **numbers** (and back). The goal is a representation that is both **meaningful** (makes sense to the model) and **small** (not too many tokens).

1. The three naive approaches (and why subwords win)

Word-based — split on spaces, one token per word.

- Problem: huge vocabulary (English has 500k+ words), can't see that "dog"/"dogs" are related, and any unseen word becomes a useless [UNK] (unknown) token.

Character-based — split into single letters.

- Tiny vocabulary and almost no unknowns, BUT each character means little on its own and one word becomes many tokens (slow, inefficient).

Subword tokenization (the winner) — keep common words whole, split rare words into meaningful pieces.

- e.g. "annoyingly" → "annoying" + "ly"; "tokenization" → "token" + "ization".
 - Small vocabulary, almost no unknowns, captures relationships. Used by all real models.
-

2. The tokenization pipeline (5 stages)

Every tokenizer is the same assembly line:

1. **Normalization** - clean the text (lowercase, strip accents, fix spaces, Unicode rules).
2. **Pre-tokenization** - rough first cut into words; this is where **offsets** are born.
3. **Model** - the subword algorithm (BPE / WordPiece / Unigram).
4. **Post-processing** - add special tokens like [CLS]/[SEP], build attention mask + token-type IDs.
5. **Decoding** - glue tokens back into readable text (reverse direction).

Normalization (example)

"Héllô hōw are ü?" → "hello how are u?" (lowercased, accents stripped) for uncased BERT.

Pre-tokenization differs per tokenizer (same text "Hello, how are you?")

- **BERT**: splits on spaces **and** punctuation; ignores the double space.
 - **GPT-2**: splits on spaces+punctuation; keeps spaces as `▯` (so it's reversible); does NOT ignore the double space.
 - **T5/SentencePiece**: marks spaces with `▯`, splits only on whitespace (punctuation stays attached); adds a leading space.
-

3. Normalization & encoding basics

Encoding = two steps:

1. **Tokenize**: text → token strings (e.g. `"Using a Transformer"` → `['Using', 'a', 'transform', '##er']`).
2. **Convert to IDs**: token strings → numbers via the **vocabulary**.

Decoding reverses it (IDs → text) and re-joins subword pieces into readable words. Always load the tokenizer from the **same checkpoint** as the model so rules + vocabulary match.

4. Fast vs slow tokenizers

- **Slow** = pure Python. **Fast** = backed by the 🦀 Tokenizers library (written in **Rust**).
- Fast tokenizers are much faster **in batches** (e.g. ~11s vs ~5min); on a single sentence you may see no difference.
- `AutoTokenizer` picks the fast one automatically. Check with `tokenizer.is_fast` → `True`.

The superpower: offset mapping

A fast tokenizer remembers, for **every token**, the exact `(start, end)` **character position** it came from in the original text. Picture a ruler under the sentence: each token knows which slice of the ruler it covers.

Helper methods on the `BatchEncoding` output:

- `tokens()` - the token strings directly.

- `word_ids()` - which **word** each token came from (special tokens → `None`; e.g. `S`, `##yl`, `##va`, `##in` all map to one word). Works for any fast tokenizer.
 - `word_to_chars()` / `token_to_chars()` / `char_to_word()` / `char_to_token()` - map between words, tokens, characters.
 - `return_offsets_mapping=True` - get the raw `(start, end)` list.
-

5. NER (token classification) by hand

Flow: tokenize → model → post-process.

- `AutoModelForTokenClassification` gives **one set of logits per token** (shape e.g. `[1, 19, 9]` = 1 sentence, 19 tokens, 9 labels).
 - **softmax** → probabilities, **argmax** → predicted label per token.
 - **Labels use the IOB scheme:** `O` = outside (not an entity); `B-XXX` = beginning of entity; `I-XXX` = inside entity. Types: PER, ORG, LOC, MISC.
 - IOB1 vs IOB2: in IOB2 every entity starts with `B-`; in IOB1, `B-` only separates two adjacent same-type entities (so a lone entity is all `I-`).
 - **Group entities with offsets:** take the text span from the first token's start to the last token's end — no brittle tokenizer-specific stitching needed.
-

6. Question answering (QA) + long contexts

- QA model outputs **TWO sets of logits**: a **start** score and an **end** score per token (it points at where the answer begins and ends).
- **Mask** non-context tokens (the question + padding) by setting their logits to **-10000** so softmax makes them ~ 0 . Keep `[CLS]` (some models use it for "no answer").
- Score every valid span: `start_prob[i] × end_prob[j]`, keep only **start** ≤ **end** (the upper triangle, via `triu`), take the best.
- Convert the winning token span → character span via **offsets**, slice the context → the answer text.

Long contexts (too many tokens for the model, e.g. >384):

- Don't just truncate — the answer might be at the end and get deleted.
- Split the context into **overlapping chunks** using `return_overflowing_tokens=True` + `max_length` + `stride` (the overlap stops the answer being cut in half).
- `overflow_to_sample_mapping` records which original sentence each chunk came from.

- Run the model on each chunk, pick the best-scoring answer across all chunks.
-

7. Training a NEW tokenizer (from an old one)

- **Training a tokenizer ≠ training a model.** A tokenizer's training is **deterministic & statistical** (find the best subwords by counting) — no gradient descent, no randomness. Same corpus + same algorithm = same result.
 - `AutoTokenizer.train_new_from_iterator(corpus, vocab_size)` copies an existing tokenizer's **algorithm + special tokens**, and only relearns the **vocabulary** on your corpus.
 - Feed the corpus with a **generator** (memory-light "tap" that yields batches, not a full list). Generators are **single-use**, so wrap in a function (or use `yield`) to get a fresh one each pass.
 - Only works on **fast** tokenizers. Example: an English tokenizer on Python code is wasteful; a retrained one learns indentation tokens, `[ ]` splits, etc., and is far more compact (27 vs 36 tokens).
-

8. The three subword algorithms

Quick memory hooks

- **BPE** = merge by **frequency** (most common pair wins). Builds UP.
- **WordPiece** = merge by **score** (most distinctive pair wins). Builds UP.
- **Unigram** = prune by **probability** (least-missed pieces cut). Trims DOWN.

BPE (Byte-Pair Encoding) — GPT-2, RoBERTa, BART

Train: start with single characters → repeatedly **merge the most frequent adjacent pair** into a new token, recording each as a **merge rule** → stop at target vocab size. **Tokenize new word:** split into characters → **replay the merge rules in order**. **Unknowns:** characters not seen in training → `[UNK]` (this is why emojis break many models). **Byte-level BPE (GPT-2's trick):** work on the **256 bytes** instead of characters. Every character is built from bytes, so the base vocab is tiny AND **nothing is ever unknown**.

Example: corpus `(("hug", 10) ("pug", 5) ("pun", 12) ("bun", 4) ("hugs", 5))` → first merge `(("u", "g") → "ug")` (20×), then `(("u", "n") → "un")`, then `(("h", "ug") → "hug")`. Tokenizing `"bug"` → `[ "b", "ug" ]`.

WordPiece — BERT, DistilBERT

- Marks continuations with `##` (`(w ##o ##r ##d)`).
- **Train:** like BPE, but picks the pair with the best $\text{score} = \text{pair_freq} / (\text{freq_first} \times \text{freq_second})$ — favors pairs whose parts are **rare on their own**, so it builds distinctive chunks instead of gluing common bits.
- **Keeps only the vocabulary** (not merge rules).
- **Tokenize new word: greedy longest-match from the front** — grab the biggest piece in the vocab starting at the front, attach `##` to the leftover, repeat.
- **Unknown rule (strict):** if it gets stuck mid-word, the **WHOLE word** $\rightarrow$ `[UNK]` (not just one character).

Example: `"hugs"` $\rightarrow$ `["hug", "##s"]`; `"bugs"` $\rightarrow$ `["b", "##u", "##gs"]`; `"bum"` $\rightarrow$ `["[UNK]"]`.

Unigram — T5, ALBERT, XLNet (via SentencePiece)

- **Opposite direction:** start with a **huge** vocabulary, repeatedly **remove the tokens whose deletion least raises a loss**, until target size. Never removes single characters (so any word is always tokenizable).
 - **Tokenize:** each token has a **probability**; score a split by **multiplying** its tokens' probabilities; pick the **most probable split** (naturally favors **fewer tokens**). The efficient search is the **Viterbi** algorithm.
 - **Loss** = summed $(-\log(P(\text{word})))$ over the corpus; pruning drops tokens that barely affect it (e.g. redundant "pu") and keeps valuable ones (e.g. "hug").
-

9. SentencePiece & Unicode/bytes (two key concepts)

SentencePiece — a toolkit (not a 4th algorithm) that runs BPE or Unigram underneath and treats text as a **raw stream of characters**, replacing spaces with `▯`. Benefits:

- **Reversible** tokenization (decode by gluing tokens and turning `▯` back into spaces).
- **No pre-tokenization needed** $\rightarrow$ works on **space-less languages** (Chinese/Japanese) by learning which character-groups go together statistically, instead of splitting on spaces.

Unicode vs bytes:

- **Unicode** = master codebook giving every character a number (A=65, 🤪=128512). Millions of characters $\rightarrow$ can't keep them all $\rightarrow$ unseen ones become `[UNK]`.
- **Byte** = a unit holding a number 0-255 $\rightarrow$ only **256 exist**, and **every** character is built from bytes. Working in bytes (byte-level BPE) means the base vocab is complete and

no input is ever unknown.

10. The practical API (using tokenizers in code)

Load / save: `(AutoTokenizer.from_pretrained(checkpoint)) / (tokenizer.save_pretrained(dir))`. A tokenizer has two parts: the **algorithm/rules** ($\approx$ a model's architecture) and the **vocabulary** ($\approx$ a model's weights).

Calling the tokenizer returns a dict-like `(BatchEncoding)`:

- `(input_ids)` - the token ID numbers.
- `(token_type_ids)` - which sentence (A=0 / B=1) each token belongs to.
- `(attention_mask)` - 1 = real token, 0 = padding (tells the model to ignore padding).

Tensors: models only accept **tensors** (grids of numbers). `(return_tensors="pt")` gives PyTorch tensors. Tensors must be **rectangular** → hence padding.

Batching, padding, truncation:

- Models expect a **batch** (even one sentence needs an outer dimension → double brackets).
- **Padding** fills shorter sentences with the pad token (ID often 0) so rows are equal length → rectangular tensor.
- **The attention_mask is essential:** without it, attention "sees" the padding and corrupts results. (Proven: same sentence gives different logits padded-in-a-batch vs alone, until you add the mask.)
- **Truncation** chops sequences over the model's max length (BERT = 512). Long inputs → use a long-context model (Longformer/LED) or truncate.

One call does it all: `(tokenizer(texts, padding=True, truncation=True, return_tensors="pt"))`.

11. Building a tokenizer block by block (🧡 Tokenizers library)

Assemble any tokenizer by choosing one block per stage. The three classic builds:

Stage	BERT (WordPiece)	GPT-2 (BPE)	XLNet (Unigram)
Model	<code>WordPiece(unk_token="[UNK]")</code>	<code>BPE()</code> (no unk)	<code>Unigram()</code>
Normalizer	NFD + Lowercase + StripAccents	none	quotes + NFKD + StripAccents + collapse spaces
Pre-tokenizer	<code>Whitespace</code> (splits punctuation)	<code>ByteLevel</code> (<code>Ġ</code>)	<code>Metaspace</code> (<code>▮</code> , whitespace only)
Trainer	<code>WordPieceTrainer</code>	<code>BpeTrainer</code>	<code>UnigramTrainer</code> (must set <code>unk_token</code>)
Post-processor	<code>[CLS] A [SEP]</code> (cls first)	<code>ByteLevel(trim_offsets=False)</code>	<code>A <sep> <cls></code> at END, type id 2)
Decoder	<code>WordPiece(prefix="##")</code>	<code>ByteLevel()</code>	<code>Metaspace()</code>
Wrap	<code>PreTrainedTokenizerFast</code> / <code>BertTokenizerFast</code>	... / <code>GPT2TokenizerFast</code>	... / <code>XLNetTokenizer</code> (pad left)

Key block notes

- **Always list special tokens in the trainer** — they aren't in the corpus, so otherwise they won't be added.
- **Post-processor template:** `($A)` = first sentence, `($B)` = second; `(:0)/(:1)/(:2)` = the token-type (segment) ID. It auto-adds the special tokens to every encode.
- `trim_offsets=False` (**GPT-2**): keeps the leading space inside a `Ġ`-token's offset (slicing `Ġtest` returns `' test'`).
- **Wrapping** = put the raw `Tokenizer` engine into a `PreTrainedTokenizerFast` shell so 🤗 Transformers can use it. The `unk_token=`, `pad_token=`, etc. arguments **label which token plays which role**, because the raw engine doesn't track roles. Use the generic wrapper for a new tokenizer; a model-specific class (e.g. `BertTokenizerFast`) inherits sensible defaults.
- **GPT-2 wrap:** `(bos)/eos` both = `<|endoftext|>` (one token marks document boundaries); no `(unk)` (byte-level can't produce unknowns).
- **XLNet wrap:** needs `(padding_side="left")` (its `<cls>` sits at the end).

12. The whole chapter in one breath

Text becomes numbers via a **5-stage pipeline** (normalize → pre-tokenize → model → post-process → decode). **Subword** tokenization beats word/character approaches. **Fast** tokenizers (Rust) add **offset mapping**, which powers NER (IOB labels) and QA (start/end spans, sliding-window chunks for long contexts). The three algorithms: **BPE merges by frequency**, **WordPiece merges by a smart score**, **Unigram prunes by probability** — with **byte-level** (no unknowns) and **SentencePiece** (`▮`, works on space-less languages) as key variants. In practice you call the tokenizer to get `(input_ids + attention_mask)`, **pad** to a rectangle (mask hides padding), and **truncate** long inputs. And you can **build any tokenizer block by block**, then **wrap** it in `PreTrainedTokenizerFast` to use it in 😊 Transformers.